

cs-mail Desktop MVP and CLI Administration

AI-Agent Milestone and Build Plan

C-SQD

Draft 0.4 — September 2026

Abstract

This plan develops a native desktop pilot, an operator administration CLI, and a reusable client core around the revised relationship-request model. Alice makes a simulated annual service purchase, then authorizes one conditional charge for a request to Bob. She sends permitted follow-ups without another charge and sees the outcome of his decision. The service then carries a simulated forfeiture through maturity, annual allocation, and one automatic bank payment with rebate/excess reporting. Each milestone requires a usable journey and evidence of failure recovery. The pilot introduces no real redeemable value.

1 Build on the Implemented Domain Contracts

The *Protocol Specification* governs requests and permission. The normative C-SQD financial-program section of the *Deployment and Migration Profile* governs annual utility billing, the pool, and equal member distributions. The *Rust Reference Architecture* describes the current ownership boundaries. This plan develops product journeys on those contracts. Start with the [C-SQD domain model](#) for confirmed rules and deferred choices. Current C-SQD accounts belong to identifiable people, one account per person, with multiple email aliases and a verified bank association. The product uses one configured payment-processing arrangement.

The workspace already implements request-specific payment operations, refunds, forfeiture maturity, annual allocation, fixed service contracts with advance collection, funded coverage, and automatic lump-sum distribution preparation. Signed service and client APIs, PostgreSQL transactions, and worker libraries exercise those rules with a simulated external provider. They are the foundation for the pilot, not a completed running server or desktop application.

The current code replaces deprecated labels and quarterly types directly. Product work must not recreate them through aliases, fallback readers, or parallel implementations. If older populated data needs conversion, handle that as an explicit data operation while preserving its real obligations. Security, privacy, ordering, and conservation remain requirements of the current model.

Product slices should reuse the existing domain owners and APIs. An interface must show the full annual service price separately from any later distribution, and one pending request must remain one charge even when follow-ups are allowed. Rates, eligibility thresholds, prices, collection dates, and distribution dates must have explicit simulation policy versions; a fixture is not an approved live-payment term.

2 The Pilot We Are Building

The initial audience is an invite-only group of approximately 20–50 cooperative users. The product is a macOS desktop application plus an administration CLI, served by one provider, one region,

and one PostgreSQL deployment. Windows follows after the macOS journey is stable. Later Swift and Kotlin clients reuse a platform-neutral Rust client core through a narrow binding.

The value model is a deterministic external-payment simulator. It can approve, decline, delay, or report an uncertain utility or request charge; confirm or fail a refund; and execute or fail a finalized member payment. All amounts are explicitly nonredeemable. Users do not receive a generally spendable test-credit wallet, and the operator configures simulator scenarios rather than topping up accounts.

Native content remains encrypted at endpoints. Devices hold revocable key authority, recovery uses user-held material or trusted-device transfer, and the provider cannot decrypt content. The desktop stores ciphertext and minimal metadata by default, with no persistent plaintext body or search index.

SMTP, independent-provider federation, public signup, live payments, a mobile release, and multi-region operation are outside this pilot. The financial simulation is included because it validates the revised business rules; it is not evidence that production payment operations are ready.

3 Keep Shared Mechanics and Product Journeys Separate

The stable foundation remains organized around primitives, authentication, ledger, finance, billing, request transitions, content, wire, storage, and workers. Product modules coordinate a user journey without implementing another version of those rules.

```
crates/  
  existing cs-mail-* crates      # current semantic foundation  
  cs-mail-api-types             # add when transport contracts need it  
  cs-mail-client                # extend existing client and billing APIs  
  cs-mail-mobile-ffi            # later portability milestone  
  
apps/  
  cs-mail-server                # API library and cs-maild  
  cs-mail-admin-cli             # authenticated operator client  
  cs-mail-desktop               # Tauri shell and platform adapters
```

Dependencies point inward: applications use the shared core, which uses the foundation; foundation crates never depend on applications. API types should not depend on a web framework, desktop shell, concrete database, or operating-system keychain. Create another crate only for a stable reusable contract, dependency boundary, or independently built artifact, not for every new feature.

Within the client and server, organize identity/device management, requests, messages, relationships, annual service billing, distributions, recovery, and administration as vertical modules. Request orchestration handles quote, authorization, content upload, payment progress, submission, and retry. Financial views consume canonical obligation status; they do not reconstruct settlement from UI events. The member program consumes eligibility and funding evidence separately from message handling. Billing owns fixed annual service contracts, collection evidence, and coverage; finance owns eligibility, distribution terms, and amounts owed. The existing application coordinator prepares payments across those two owners. Product controllers must not duplicate that calculation.

The client core defines ports for key custody, local ciphertext storage, transport, clock, and notifications. Tauri, Swift, Kotlin, Keychain, SQLite, and concrete HTTP libraries belong in adapters. Use constrained, versioned DTOs and opaque canonical signed bytes for protocol mutations. An

HTTP handler cannot translate a signed command into a different meaning.

4 Contracts That Apply to Every Milestone

Endpoint private keys use operating-system secure storage and never enter logs, configuration, SQLite, provider APIs, or diagnostics. Device enrollment and revocation require authenticated authority. Administrative credentials are distinct from user credentials, and the CLI invokes an authenticated API rather than writing canonical tables directly.

Every visible outcome has a durable cause and stable retry identity. Unknown payment or commit outcomes remain distinguishable from failure. UI language separates preparation for submission, waiting for a recipient decision, a refund due, refund completion, finalized member allocation, and payment completion. Annual service coverage and charge status have their own view. A distribution does not mark the service year unpaid, and a nonrenewing member can still inspect an allocation and its payment status. No screen suggests that rejecting a sender pays the recipient or that a payable can fund another request.

Use separate protocol, wire, API, event, policy, and database versions. Migrations are forward-only after merge. Logs contain safe scoped references and typed errors, not plaintext, secrets, identity mappings, full signed commands, or complete financial snapshots. Retention policy must be explicit before pilot data is created, including records needed for unsettled simulated obligations.

At each implementation handoff, run the workspace quality gate:

```
cargo fmt --all -- --check
cargo clippy --workspace --all-targets -- -D warnings
cargo test --workspace
```

PostgreSQL integration tests run against an isolated database; ignored tests do not count as evidence. Add contract, migration, desktop smoke, and recovery tests as the relevant surfaces appear.

5 Milestone Map

Stage	User-visible result	Exit evidence
Baseline	Implemented domain contracts	Current conformance and PostgreSQL tests
0	Product foundation is ready	Buildable skeleton and dependency checks
1	Operator starts the service	Bootstrap, simulator, CLI, and restart demo
2	Alice owns an identity and funded service	Enrollment, annual charge, coverage, and reopen demo
3	Alice requests a relationship	One conditional charge and encrypted submission
4	Bob controls the request	Decisions, follow-up policy, and accurate activity views
5	Time and failure are recoverable	Deadline, provider-event, and crash matrix
5b	The member cycle is demonstrable	Annual allocation and one bank payment with recovery
6	A small pilot can use it	Signed builds, recovery, restore, and usability evidence
7	The core is portable	Swift/Kotlin binding parity harness

Each stage depends on the preceding one. A milestone is complete when its demonstration runs from a clean checkout and its relevant failure cases pass, not merely when the new code compiles.

6 Milestone 0: Establish the Product Foundation

Using the existing semantic baseline, create the product packages and enforce inward dependencies. Define shared package metadata, lints, and dependency versions. Document the architecture boundary and add a command that runs the full quality gate without hiding individual checks. Empty server and CLI binaries should report version and help.

Demonstrate the workspace build and dependency graph from a clean checkout. The exit evidence is a passing baseline suite, reviewed dependency rules, and a small package skeleton. No enrollment, API listener, desktop workflow, or further semantic change is part of this stage.

7 Milestone 1: Bring the Service and Simulator to Life

The operator can migrate an empty database, bootstrap one provider idempotently, start the API, create invites, inspect policy, and select simulation scenarios without editing SQL. Use Tokio, Axum, Rustls, structured tracing, and Clap as the initial process, HTTP, TLS, diagnostics, and CLI choices. Keep synchronous storage behind a bounded blocking boundary until a tested replacement is appropriate.

Configuration identifies the deployment domain, provider authority, public URL, database, content limits, retention versions, and simulation mode. Expose versioned request pricing and immutable annual service offers through the authorized administration surface, including funding-source verification and attempt limits. Ordinary administration uses an authenticated API. A local bootstrap/migration command may initialize the deployment, but cannot silently replace an existing identity or key registry.

The simulator persists operation identities and supports controlled delayed, duplicate, out-of-order, and failed results. Its scenario changes are audited and separate from canonical financial writes. Reuse the existing provider model, adding durable scenario controls at the host boundary. No live provider credentials or redeemable amounts are enabled.

Tests cover migration from empty, repeated bootstrap, missing keys, user/admin authority separation, invite replay, simulator-operation idempotency, and readiness that distinguishes process health from database readiness. Demonstrate restart without duplicate authority or simulated operations. Exit evidence includes a clean-start transcript, API contracts, database tests, and redacted diagnostics.

8 Milestone 2: Establish Identity and Annual Service

Alice installs the macOS application, redeems an invite, establishes a simulated verified person/bank association, and creates protocol identities and device keys for her one account. Multiple email aliases do not create more accounts or shares. She reopens the same account after quitting. Portable client use cases coordinate enrollment and device management; the Tauri shell remains thin.

Alice reviews a published offer with annual price, advance collection date, and fixed year-long service interval. Collection occurs well before the interval; finalized funding enables coverage only within it. Signed account actions use active operational keys with explicit account grants and a separate signing context, not a second user-managed billing secret. Trusted operator configuration publishes offers and establishes verification authority. A pending charge does not imply renewed service.

Test explicit calendar and leap-day handling, failed and late collection retries that preserve the same price and interval, and coverage boundaries. Restart must preserve the attempt identity

and result. Duplicate person registration and ungranted account actions must fail. Restrict a source after work is queued and show that first dispatch waits for authenticated re-verification; reconciliation of an already-authorized uncertain operation retains its original identity.

Store only key references, ciphertext, cursors, minimized relationship metadata, and nonsecret preferences in SQLite. Use the macOS keychain for secrets. Explain the recovery capabilities that are available at this stage without implying provider access to old content.

Tests reject foreign-scope or revoked device authority, conflicting invite reuse, and secret leakage to storage or logs. Local schema upgrades preserve ciphertext and cursors. Demonstrate enrollment, keychain denial, quit/reopen, and device listing and a funded annual service interval on a clean user account. Sending and completed recovery remain deferred.

9 Milestone 3: Submit One Encrypted Relationship Request

With funded service available, Alice resolves Bob and reviews C , S , the total conditional charge, the request deadline rule, and possible refunds. The UI identifies the value as simulated. She authorizes one request, encrypts the initial message, and obtains signed evidence of submission after the simulator finalizes funding and delivery preparation is durable.

The client state machine persists the request and operation references across quote, create, payment preparation, ciphertext upload, and submission. An existing request in preparation or awaiting a decision is resumed instead of charged again. No delivery is visible before durable admission. Bob receives an encrypted inbox record and decrypts at his endpoint.

Test two-device creation with distinct command keys, capture failure, uncertainty, expired terms, wrong scope, modified declarations, ciphertext tampering, and network loss at each boundary. A declined charge admits nothing; an unknown charge is reconciled under its original identity. Alice's request activity must match the ledger and receipt, without a spendable balance view.

Demonstrate the complete encrypted path and recovery after an interrupted charge response. Exit evidence is an end-to-end request test, a payment-step retry matrix, and plaintext-absence checks. Attachments, search, rich text, SMTP, and real currency remain deferred.

10 Milestone 4: Make Bob's Decision and Follow-Up Controls Real

Bob sees one request with its grouped messages and a clear decision deadline. Acceptance grants future communication and records one full refund obligation; rejection records processing revenue and pending forfeiture; blocking additionally prevents contact. The UI explains permission first, then money, and never describes rejection as a payment to Bob.

Expose the recipient's collateral menu and default with authenticated preference updates. Changing a selection affects the next quote, leaving already authorized requests intact. Implement recipient follow-up preferences with bounded count/rate controls and canonical admission enforcement. Alice can add a message only when the identified request and current policy allow it. Its own delivery state remains visible, but the charge and common deadline remain unchanged. Closing or dismissing the recipient's prompt is not a decision.

Show request charges and refund status separately from member distribution statements. Accepted sending uses standing permission rather than a zero-value quote. Block/unblock management preserves the difference between rejection and prohibition, and projection rebuild after restart reproduces canonical state.

Tests cover one request with three messages and one refund, disabled follow-ups, allowance replay, policy-change races, decision races, alias history, and block before versus after submission. A follow-up refusal must not start another charge. Demonstrate accept, reject, block/unblock, and allowed/disabled follow-ups from both clients. Exit evidence includes reviewed financial wording and encrypted accepted-message delivery.

11 Milestone 5: Recover Across Time and Failure

Run scheduled request deadlines, outbox delivery, payment reconciliation, and retention through supervised leased workers. Jobs recheck current state and use stable operation identities. Configure a distinct scheduler authority, bounded retries, graceful shutdown, database TLS, and a bounded connection pool without changing the revised semantics.

This milestone makes uncertainty visible. Kill the service after capture but before its response is recorded; cancel a request during preparation before delayed funding confirmation; replay provider notifications out of order; and fail a refund before retrying it. The request stays cancelled when appropriate, all captured money is returned once, and a terminal obligation does not disappear during recovery.

Also expire service coverage while a request and member obligation remain unsettled. New communication must stop, but the recipient can still decide and the service can finish refunds and payments. Reconciliation can continue querying an uncertain bank operation; it must not initiate unbounded new funding attempts.

Exercise timely acceptance delayed behind expiry processing and late acceptance after a closed window. Preserve current permission while materializing old financial outcomes. Offline recipients and cursor replay must not duplicate delivery or financial effects. Retention must protect content still needed by undelivered work while eventually deleting eligible records.

Demonstrate restart, lease reclaim, deadline processing, delayed-provider recovery, and one visible message/economic outcome after retries. Exit evidence is a failure-injection matrix, scheduler-authority tests, a sustained concurrency run, and an operator reconciliation procedure.

12 Milestone 5b: Complete an Annual Member Cycle

The operator demonstrates how rejected requests eventually benefit members without paying their recipients directly. Use the existing finance domain to create simulated lots with different maturity and quoted-rate versions, and a membership set with explicit activity and verification evidence. Fixture policies remain visibly provisional where live eligibility or identity checks are unsettled.

Finalize a UTC calendar year from fixed funding and eligibility inputs. Show the once-only corporate assessment, equal minor-unit allocations, and restricted remainder in aggregate reconciliation and individual statements. Include zero members, empty funding, exact cutoff boundaries, mixed rate cohorts, immature funds, and replay. A following year must not reassess old funds or divide amounts already assigned to members.

Next demonstrate one lump-sum payment of each member's previous-calendar-year share. Distribution terms set the due date independently of service collection. Ordinarily the receiving service year is already paid. Using $U = \$120$, exercise separate allocation scenarios of \$80, \$120, and \$170: each produces one payment, with rebate/excess reporting of \$80/\$0, \$120/\$0, and \$120/\$50. These are examples across scenarios, not unequal shares for equally qualifying members of the same pool year.

Use the application coordinator to resolve the associated verified bank account automatically. No per-distribution instructions or payout minimum are supplied. Keep next-year service collection pending and show that distribution proceeds unchanged. Repeat with nonrenewal and closure, preserving the minimal payment route. Fail or lose the response to the single bank payment, restart, and prove that recovery does not duplicate it or create independent component work. Distribution leaves purchased coverage intact. Delayed obligations remain assigned even across a later allocation year.

Exit evidence includes reproducible equal allocation, automatic due-time bank payment, unchanged reporting after nonrenewal, closure recovery, and one payment completion status. Statements separate the full service charge from the later distribution. Live payments and production verification remain deferred.

13 Milestone 6: Run a Small Supported Pilot

Package and sign the macOS application and versioned server/CLI artifacts. Complete user-held recovery and trusted-device enrollment, with explicit device revocation and no provider content-decryption authority. Add understandable onboarding for annual service charges, requests, refunds, simulated distributions, follow-up preferences, encryption, recovery, and device loss.

Provide scoped diagnostics export, support and abuse intake, deletion procedures, backup and restore, incident escalation, and release rollback instructions. Backups and retained obligation records must respect separate keys and access roles. Windows adapters follow only after the core macOS acceptance path is stable and platform-neutral.

Tests cover clean installation and upgrade, recovery on a new device, revoked device refusal, diagnostics secret scans, and restore followed by reconciliation of fixed service contracts, collection attempts, requests, refunds, annual allocations, payables, and inbox state. Run a 50-identity soak with offline periods, decisions, retries, and restarts.

Give nondeveloper testers clean machines and the pilot guide. They should enroll, request a relationship, use follow-up settings, decide, inspect financial status, recover a device, and export safe diagnostics without direct database repair. Exit evidence includes signed releases, restore and recovery records, and resolved critical usability findings. Public signup and production financial value remain outside this milestone.

14 Milestone 7: Prove Mobile Portability

Expose a narrow FFI facade for identity, request progress, decisions, encrypted inbox, annual billing and coverage, financial status, distribution statements, and device management. It uses bounded values, stable error codes, cancellation, and explicit ownership rather than exporting internal domain structs or secret bytes.

Generate Swift and Kotlin bindings and provide mock platform adapters for secure keys, transport, ciphertext storage, and notifications. Run the same client-core stories through Rust and each binding, including a pending refund and failed member payment. Malformed input returns typed errors rather than panics, and process termination cannot leave an unknown operation represented as completed.

The exit evidence is a versioned facade, clean binding builds, and behavior parity without Tauri in the shared dependency graph. Store releases, mobile push, and a polished mobile UI are separate work.

15 How an Implementation Agent Should Proceed

For each slice, read the applicable revised contracts and local instructions, name one user-visible outcome, identify its trust and persistence boundaries, then implement contracts, domain orchestration, adapters, and UI in that order. Use a real end-to-end path early; do not build every route before demonstrating one request. Report what changed, what was demonstrated, and what remains open.

Do not bypass signed execution, write canonical tables from the CLI, persist plaintext for convenience, hide an uncertain charge, reinterpret old migrations, or weaken a conformance test merely to finish a milestone. Changes to the approved baseline require an explicit decision and corresponding new evidence. Scope must not expand to live value or SMTP simply because an adapter is available.

If reconciliation fails, preserve the affected evidence and investigate the specific obligation. If secrets leak, stop accumulating affected user data until the boundary is repaired. If a policy parameter is unresolved, use an explicitly versioned simulation fixture and keep the live deployment gate closed rather than presenting that fixture as a business commitment.

16 Completion and the First Assignment

The pilot is complete when clean installations can enroll and recover identities, exchange encrypted relationship requests and accepted messages, enforce follow-up preferences, survive deadlines and failures, and reproduce an annual allocation and member payment cycle with correct unpaid obligations. The operator can deploy, reconcile, restore, and support it without arbitrary database edits. The published pilot policy states simulation limits, retention, abuse review, and deletion behavior.

The next assignment is Milestone 0 and the smallest service/CLI path over the existing libraries. It should start the service, establish authenticated administration, and expose a durable simulated annual charge. Later milestones extend that path through encrypted requests and the annual member cycle without reimplementing the domain rules.